Intelligenza Artificiale

“Markov Decision Processes (part 1)”

Nicola Bellotto

A.A. 2025/2026

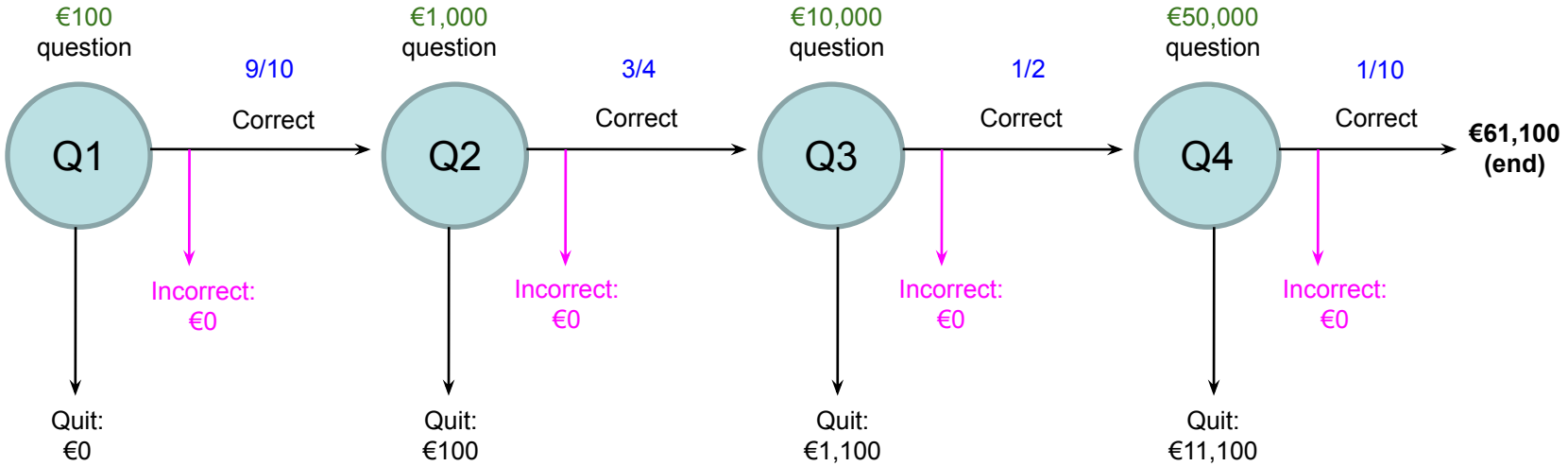
Lecture outline

- Sequential decision problems
- Utility theory
- Markov Decision Process
- Value iteration algorithm
- Application example

Slides partly based on Russell & Norvig instructors material http://aima.cs.berkeley.edu/instructors.html

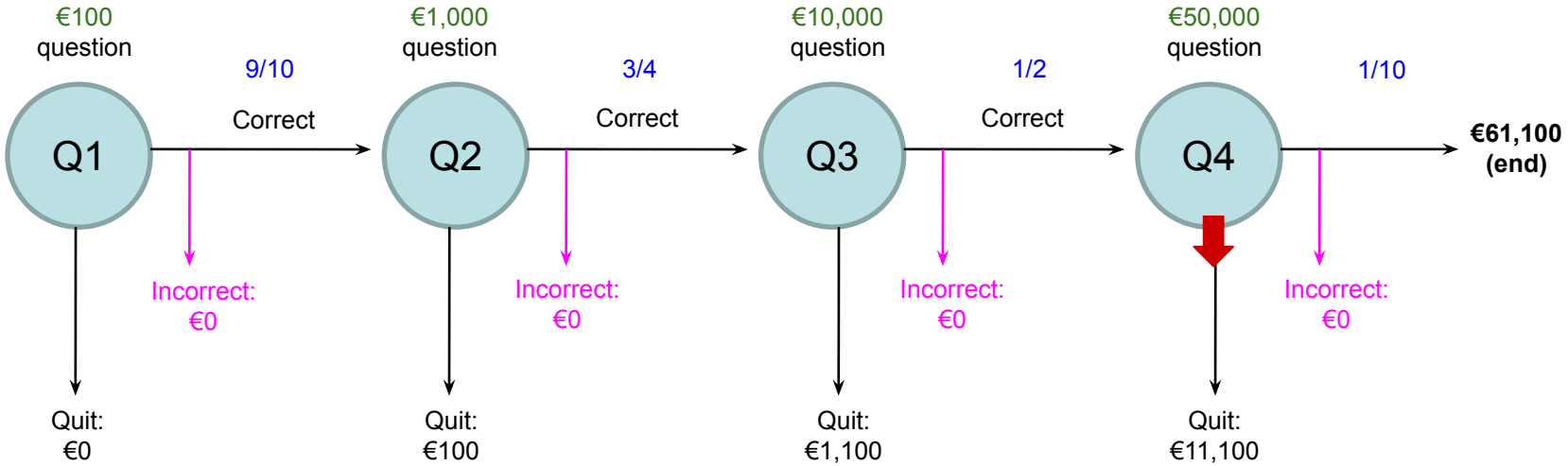
Example of sequential decision problem

- A series of questions with increasing level of difficulty and increasing payoff
- At each step, two possible **actions**: 1) take your earnings and quit, or 2) go for the next question – if wrong, lose everything!
- Questions with increasing difficulty and increasing reward



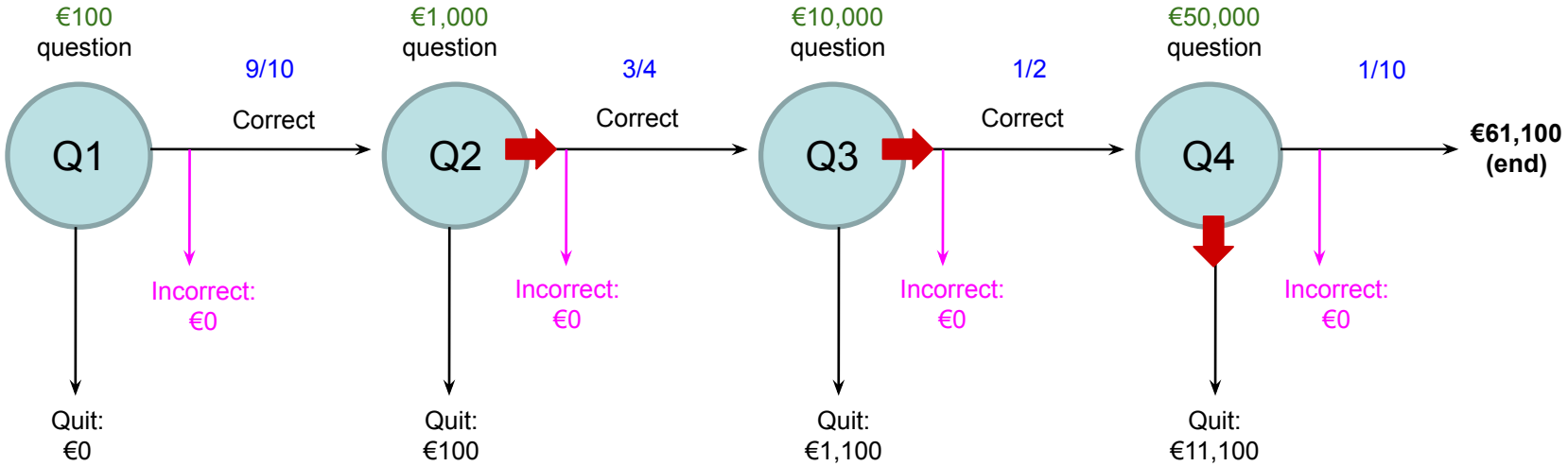
Example of sequential decision problem

- Consider question **Q4** ($Reward = €50{,}000$, $P_{Correct} = 1/10$)
  - Quit or go for the question?
- Potential earnings after 4 correct answers: $100 + 1{,}000 + 10{,}000 + 50{,}000 = €61{,}100$
- What is the expected payoff for continuing? $0.1 * 61{,}100 + 0.9 * 0 =$ **€6,110** (< **€11,100** quitting!)



Example of sequential decision problem

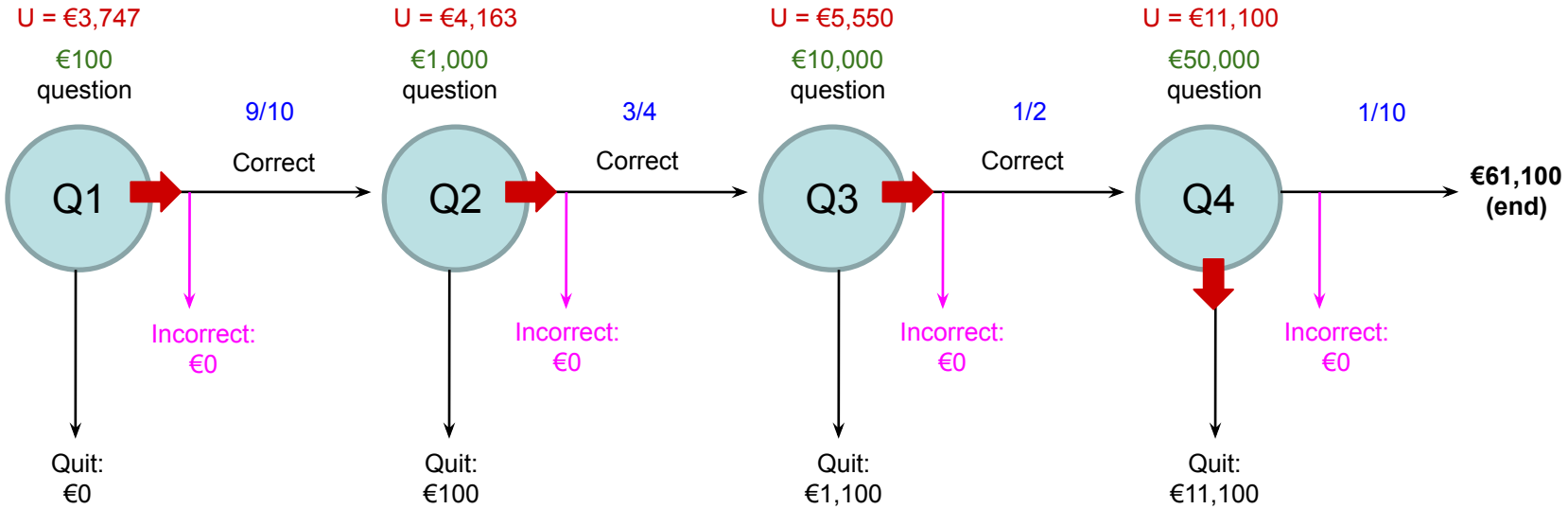
- What should we do in **Q3**? ($P_{Correct} = 1/2$)
  - payoff for quitting: **€1,100**
  - for continuing: 0.5 * 11,100 = **€5,550**
- What about **Q2**? ($P_{Correct} = 3/4$)
  - **€100** for quitting
  - 0.75 * 5,550 = **€4,163** for continuing



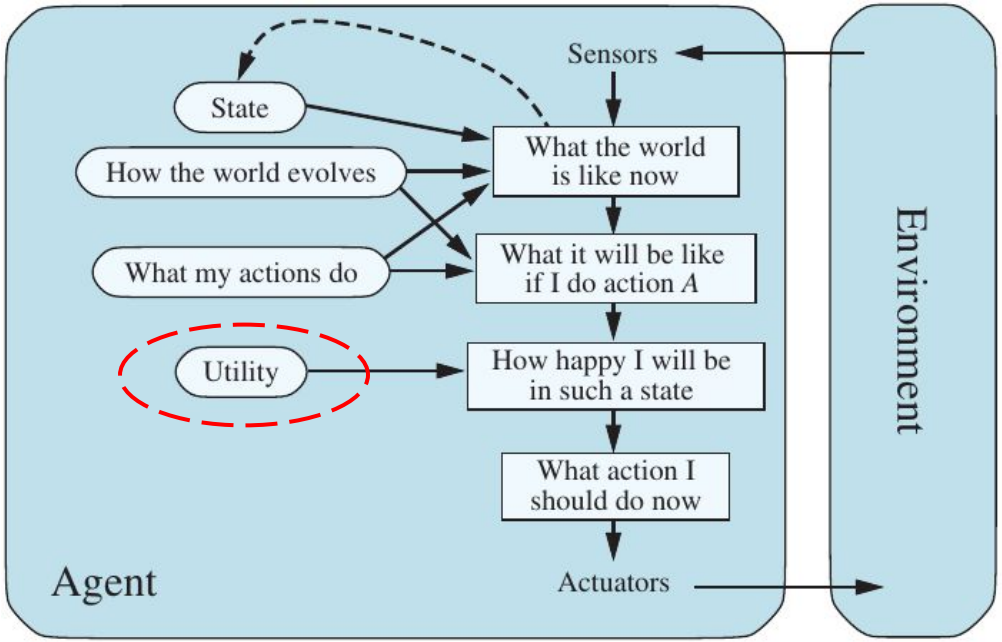
Example of sequential decision problem

- What about **Q1**? ($P_{Correct} = 9/10$)
  - **€0** for quitting
  - $0.9 * 4{,}163 =$ **€3,747** for continuing

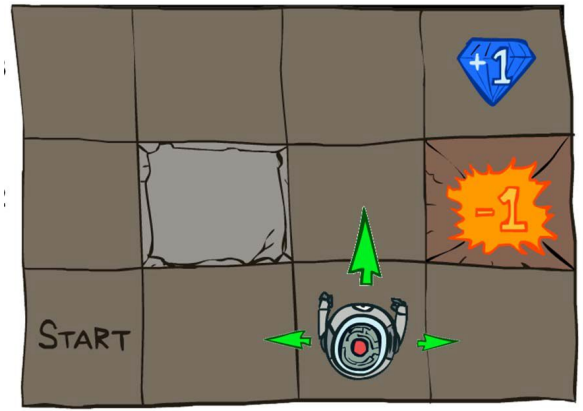
- The money potentially earned at each state (question) is the expected **utility** of that state, under the assumption that we always take the best (most rational) action



Utility-based agents



- Examples
  - an agent solving a quiz game
  - an autonomous trading platform
  - a robot that has to to reach a goal, avoiding dangers…



Expected utility

- Let's call $U(s)$ the **utility** of being in a certain state s
- $U(s)$ is a sort of numerical "score" assigned to s – i.e. how *desirable* is to reach that particular state (e.g. money earned in the quiz game)
    - Usually normalised between 0 and 1
- The **expected utility of an action**, $EU(a)$, is the average utility of the outcomes, weighted by the probability that the outcome occurs:

$$EU(a) = \sum_{s'} P(\text{RESULT}(a) = s')\ U(s')$$

- We look for a sequence of actions (**policy**) that maximize the expected utilities
    - usually simplifying a problem by recursively breaking it into smaller pieces and remembering their optimal solutions

Class of Markov models

Markov models	Passive (observations only)	Active (possible actions)
Fully observable state	**Markov chain**	**MDP** Markov Decision process
Partially observable state	**HMM** Hidden Markov Model	**POMDP** Partially Observable MDP

Markov Decision Process

- **Markov Decision Process** (**MDP**): a sequential decision problem for a fully observable, stochastic environment
- MDP consists of:
  - a set of **states** (with an initial state s_0)
  - a set of **actions** in each state, $\text{ACTIONS}(s)$
  - a **transition model** $P(s' \mid s, a)$
  - a **reward function** $R(s, a, s')$

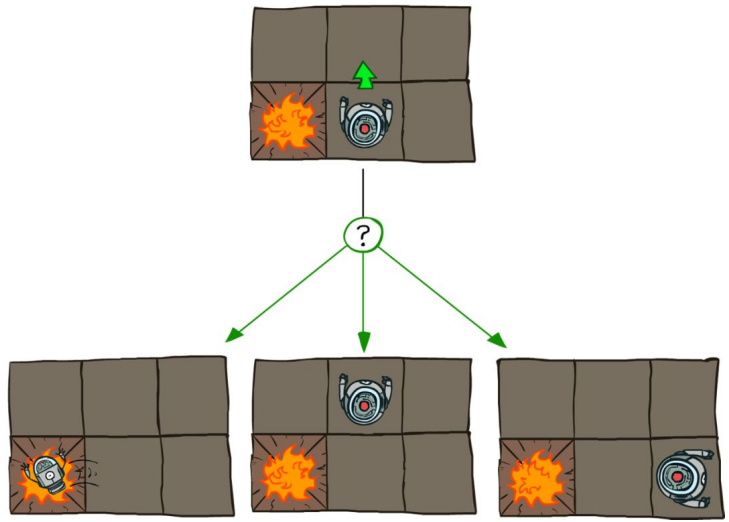
Transition model

- An agent assigns a probability $P(s)$ to each possible current **state** s
- There may also be uncertainty about the **action** outcomes

EXAMPLE

★ We might give the command "**GO UP**" to the robot, but for some reason (e.g. slipping wheels) it turns left or right instead

  - Transition probability that it goes up, given starting position + command GO UP, is **0.8**
  - Transition probability it turns left or right, given starting position + command GO UP, is **0.1**



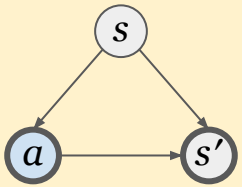
Transition model

- The **transition model** $P(s' \mid s, a)$ is used to compute $P(\text{RESULT}(a) = s')$, i.e. the probability of reaching s' doing action a from <u>any</u> state s is

$$P(\text{RESULT}(a) = s') = \sum_s P(s)\, P(s' \mid s, a)$$

  - This probability is the one necessary to compute the action's expected $EU(a)$

Note similarity to <u>adjustment formula</u> used in causal inference for calculating $P(s' \mid do(a))$



Reward function

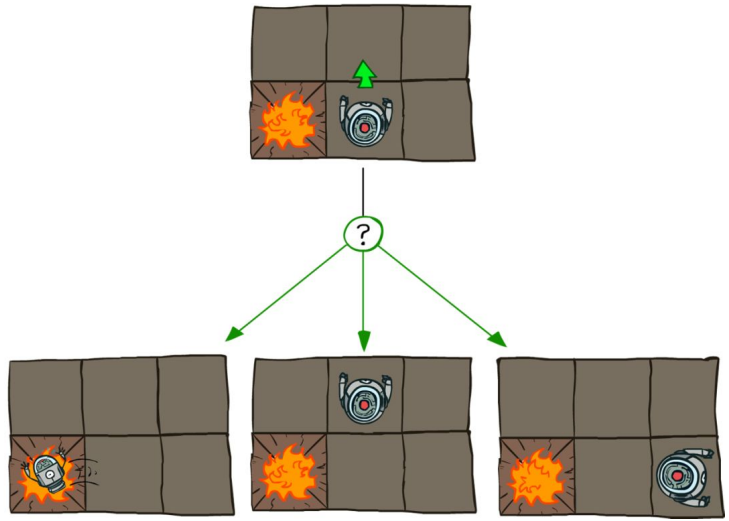
- A number (positive or negative) assigned to each transition from s to s' via action a

  ⚠️ **ATTENTION**: reward il local, utility is global (takes into account future states)

EXAMPLE

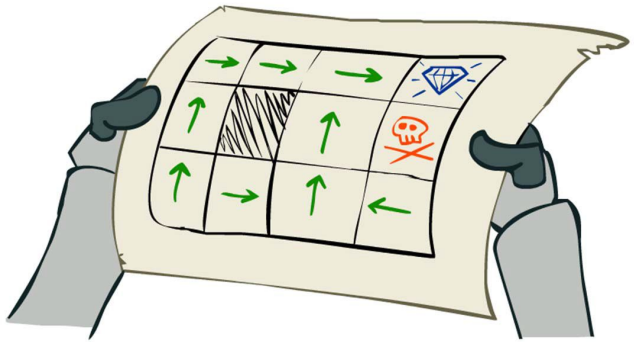
★ In the robot grid-world

- moving to the fire cell might have reward –1000
- moving to any other cell might have reward 1



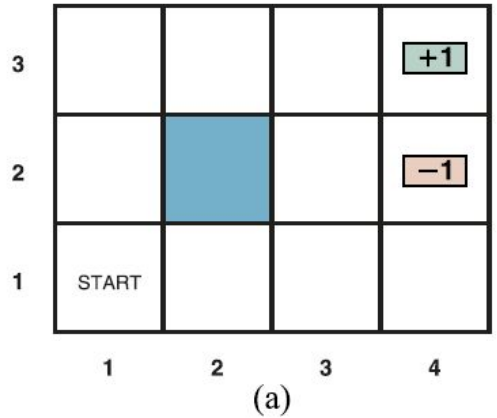
Policy

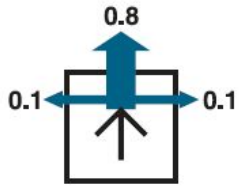
- A solution for the MDP is a **policy**
  - specify what the agent should do for any state that the agent might reach



- The quality of a policy is measured by the expected utility of possible action sequences
- **Optimal policy** = sequence of actions that brings the highest expected utility

Policy example



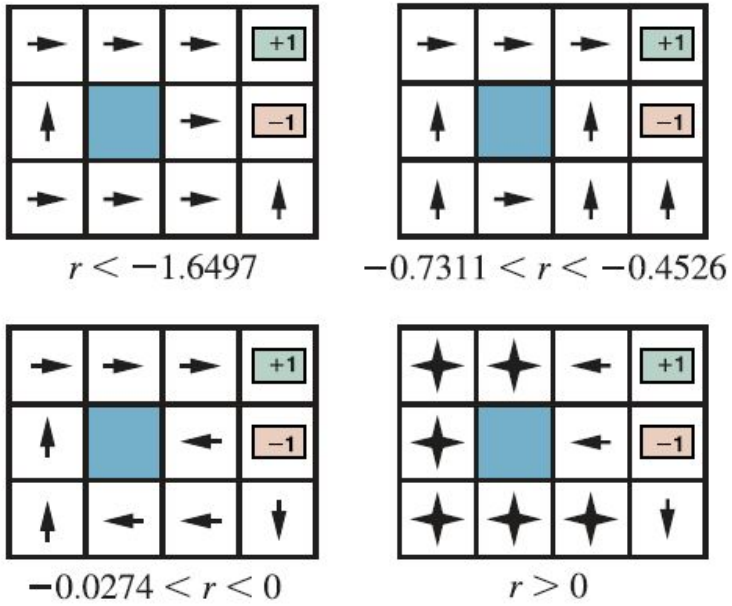


(a) A simple stochastic 4x3 environment that presents the agent with a sequential decision problem

(b) Transition model: the “intended” outcome occurs with probability 0.8, but with probability 0.2 the agent moves perpendicularly (left or right) to the intended direction

- A collision with a wall results in no movement
- Transitions into the two **terminal states** have reward +1 (good!) and –1 (bad!)
- All other transitions have a **reward** r

Policy example



- Optimal policies for four different ranges of r

Utilities over time

Utilities take into account future states – two possible cases:

- **Finite horizon**: fixed time N after which nothing matters
  - optimal action in a given state may depend on how much time is left
  - ⇒ optimal policy is **nonstationary** (depends on time)

- **Infinite horizon**: no fixed time limit
  - no reason to behave differently in the same state at different times
  - optimal action depends only on current states
  - ⇒ optimal policy is **stationary**

we focus on this

Bellman equation

- **Bellman equation** – the utility $U(s)$ is the expected reward for next transition + discounted utility of next state, assuming the agent chooses the optimal action
  - γ is a number between 0 and 1 called **discount factor**
    - γ close to 1 → willing to wait long-term reward
    - γ close to 0 → not willing to wait

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s' \mid s,a)[R(s,a,s') + \gamma U(s')]$$

solved by iterative process

- The sum to be maximised is the expected utility of taking a given action in a given state, called **action-utility function**, or **Q-function**: $Q(s, a)$

$$U(s) = \max_{a} Q(s,a)$$

Utility example

3	0.8516	0.9078	0.9578	+1
2	0.8016		0.7003	−1
1	0.7453	0.6953	0.6514	0.4279
	1	2	3	4

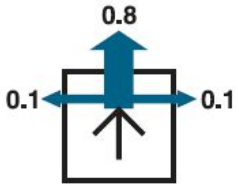
- The utilities of the states in the 4x3 world with $\gamma = 1$ and $r = -0.04$ for transitions to nonterminal states
- In $(1, 1)$, the best action would be "***Up***"

$$U(1,1) = \max\{\mathbf{[0.8(-0.04 + \gamma\, U(1, 2)) + 0.1(-0.04 + \gamma\, U(2, 1)) + 0.1(-0.04 + \gamma\, U(1, 1))]}, \ (\textit{Up})$$

$$[0.9(-0.04 + \gamma\, U(1, 1)) + 0.1(-0.04 + \gamma\, U(1, 2))], \quad (\textit{Left})$$

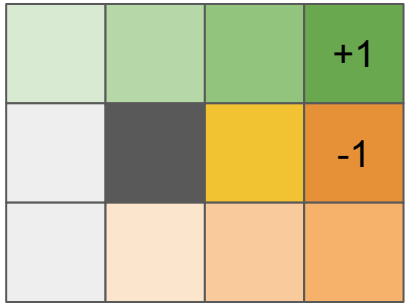
$$[0.9(-0.04 + \gamma\, U(1, 1)) + 0.1(-0.04 + \gamma\, U(2, 1))], \quad (\textit{Down})$$

$$[0.8(-0.04 + \gamma\, U(2, 1)) + 0.1(-0.04 + \gamma\, U(1, 2)) + 0.1(-0.04 + \gamma\, U(1, 1))]\} \quad (\textit{Right})$$



Value iteration algorithm

- Bellman equation is the basis of the **value iteration** algorithm
  - If there are n possible states, then there are n Bellman equations
  - The n equations contain n unknowns – i.e. the utilities U of the states
  - The equations are nonlinear because of the "max" operator



💡 **IDEA**: start with arbitrary initial utilities, calculate the equation **recursively**, updating the utility of each state <u>from the utilities of its neighbors</u>

$$U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s' \mid s,a)[R(s,a,s') + \gamma U_i(s')]$$

  - Repeat this until an **equilibrium** is reached

Value iteration algorithm

- Update is applied simultaneously to all the states at each iteration
- The algorithm stops when the difference δ between old and updated utilities is below a certain threshold
- The output utility can then be used to compute the optimal policy:

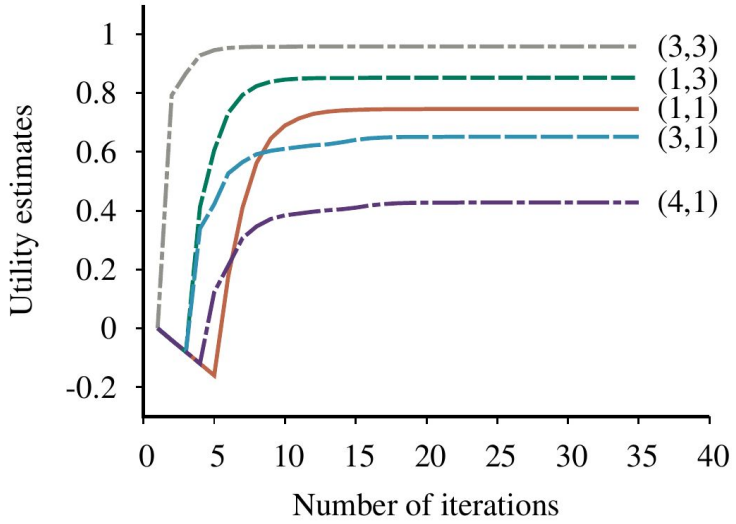
$$\pi^*(s) = \operatorname*{argmax}_a Q(s,a)$$

function Q-VALUE(mdp, s, a, U) **returns** a utility value
  return $\sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U[s']]$

function VALUE-ITERATION(mdp, ϵ) **returns** a utility function
  inputs: mdp, an MDP with states S, actions $A(s)$, transition model $P(s'|s,a)$,
    rewards $R(s,a,s')$, discount γ
    ϵ, the maximum error allowed in the utility of any state
  local variables: U, U', vectors of utilities for states in S, initially zero
    δ, the maximum relative change in the utility of any state

  repeat
    $U \leftarrow U'$; $\delta \leftarrow 0$
    for each state s **in** S **do**
      $U'[s] \leftarrow \max_{a \in A(s)}$ Q-VALUE(mdp, s, a, U)
      if $|U'[s] - U[s]| > \delta$ **then** $\delta \leftarrow |U'[s] - U[s]|$
  until $\delta \leq \epsilon(1-\gamma)/\gamma$
  return U

Value iteration – example



3	0.8516	0.9078	0.9578	+1
2	0.8016		0.7003	−1
1	0.7453	0.6953	0.6514	0.4279
	1	2	3	4

Evolution of the utilities of some states using value iteration in the 4x3 world

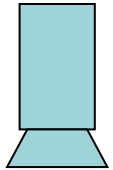
Application example

Automatic hand-washing guidance for people with dementia

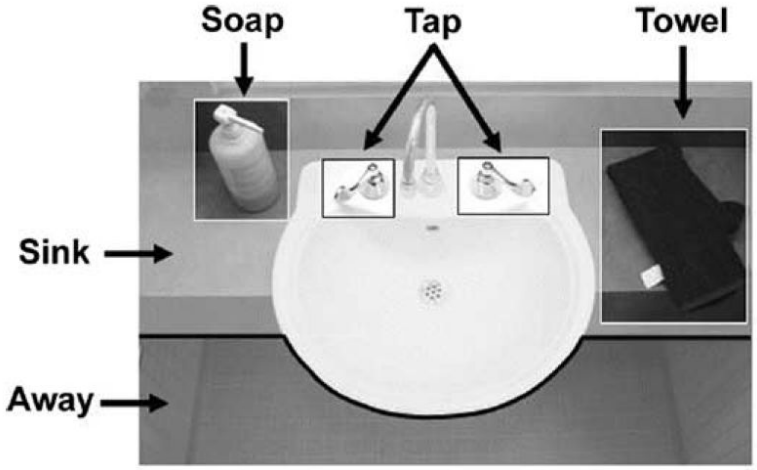
- People with dementia have great difficulty in executing sequential tasks such as washing hands, preparing a cup of tea, etc.
- J. Boger et al. (2006) developed an automatized system based on MDP can guide the person in executing a task to reach the goal (washed hands) with maximum probability

J. Boger et al. (2006) “A Planning System Based on Markov Decision Process to Guide People with Dementia Through Activities of Daily Living”, IEEE Trans. Information Technologies in Biomedicine.

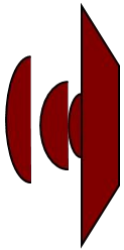
System overview



camera



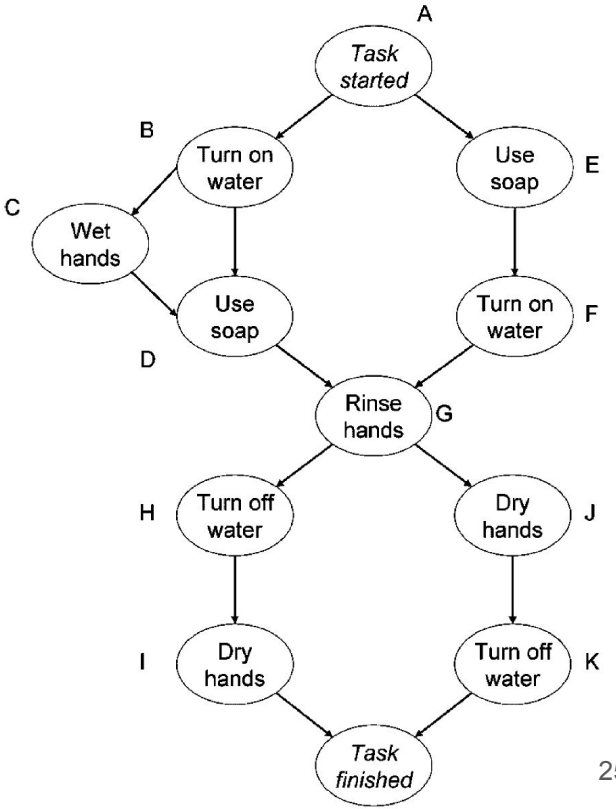
speaker



Problem definition

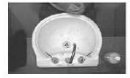
- MDP components:
  - **States**: hands position, water flow, number of prompts, number of regressions, etc. (> 20M states!)
  - **Actions**: 20 possible instructions (through speaker), with varying level of detail
  - **Transition model**: defined by an expert caregiver/doctor (but could be learned from data)
  - **Reward function**: depending on the particular sequence of states and actions
- The solution:
  - **Policy**: mapping from states to actions to complete the hand-washing task successfully

Desired state sequence



Execution

- The task starts with the user in front of the sink
- The system generates a vocal instruction (i.e. system's "action") depending on the current stage of the process
- The instruction is the most likely one leading to the next desired state, until hands have been washed and dried

Time(s)	User status	PS	Prompt
0		A	Time to wash your hands, John
6		B	
14		B	Put some soap on your hands
37		D	Rinse your hands under the water
55		G	
71		G	Pick up the towel on your right
80		D	John, rinse you hands
104		G	Use the towel on your right
170		K	Thank you John, we're all done here

Conclusions

- Suggested reading
  - Chapter 15, sec. 15.1 of Russell & Norvig
  - Chapter 16, sec. 16.1, 16.2 of Russell & Norvig

- Next lecture

  Markov decision processes (part 2)

- Questions?